

10th International Conference of Information and Communication Technology (ICICT-2020)

Convolutional neural network based dictionary learning to create hash codes for content-based image retrieval

Şaban Öztürk*

Amasya University, Technology Faculty, Amasya, 05001, Turkey

Abstract

This study investigates the suitability of sparse vectors in the dictionary learning (DL) method for content-based image retrieval (CBIR) tasks. Since DL usually performs the learning process in an unsupervised manner, it cannot generate robust features for the retrieval task, especially if a complex background is involved. In order to overcome this drawback, a DL approach using the feature representation power of the convolutional neural network (CNN) is proposed. The initialization values of the dictionaries in the proposed CNN based DL method are taken randomly from the middle layers of the CNN architecture. The vector of each image obtained from the CNN architecture is used as DL input. The lambda vectors produced by the DL structure are converted into binaries. In this way, DL acts as a hash code generator. The performance of the proposed framework is tested on modified COREL dataset. The results prove that it is an open-to-improvement approach and is promising.

© 2021 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the 10th International Conference of Information and Communication Technology.

Keywords: CNN; Dictionary learning; sparse coding; CBIR, hashing.

1. Introduction

The nearest neighbor (NN) search is the approach to offer the most similar features to query features in a large dataset. This is done using problem-specific distance metrics¹. It is very costly to use exact NN due to the increase in the data obtained from real applications. For this reason, using approximate nearest neighbor (ANN) is enough and useful to solve many practical problems². Hashing is a widespread solution for ANN, lately³. It converts the original data into binary format and maps to a low-dimensional format. These vectors, which are named Hash code, maintain

* Corresponding author. Tel.: +90-506-570-2452; fax: +90-506-570-2452.

E-mail address: saban.ozturk@amasya.edu.tr

the similarity with the original space. Hashing has two major advantages. Storage cost is greatly reduced thanks to binary hash code representations. Additionally, the retrieval speed is greatly increased⁴.

Feature extraction and vector applications can be considered as the two most important steps of generating hash code. Descriptive vectors generated in the feature extraction step are essential for high-performance retrieval tasks. On the other hand, in the vector editing step, the size of the vector is reduced, and binarization processes are carried out. This step determines the storage capacity and retrieval speed⁵. While features were extracted using hand-crafted methods in the past⁶, nowadays, image features are obtained with CNN methods⁷. CNN algorithms automatically extract the features of the images and can also classify these features. Using different CNN architectures, operations such as image segmentation, object detection, regression, and tracking are performed. They are used in almost all image processing works today thanks to their feature representation power. CNN-based frameworks for generating hash codes in CBIR studies are quite common recently⁸. In CBIR studies performed with CNN architecture, two approaches are generally used for hash code generation: end-to-end architecture using CNN output and multi-stage methods to generate hash code using CNN features. End-to-end methods generally focus on loss functions. Although triplet-loss based methods are quite efficient, computational complexity is quite high⁹. Besides, it requires a highly comprehensive tagged dataset. Triplet-wise, pair-wise and point-wise methods usually require more than one loss function^{10,11}. Computational complexity increases as the number of parameters that need to be optimized for these methods is included in the loss functions. Although end-to-end CNN methods are very useful for hash code generation, their performance still needs to be improved. In the multi-stage hashing approaches, another method is fed using features obtained from the CNN algorithm and the hash code is created¹². Clustering the features in the CNN output with k-means clustering and coding with the VLAD descriptor is a good example of these approaches¹³. Although the performance of these methods is quite remarkable, it is a handicap to include more than one step. It is difficult and some approach are complex to train. They may not be considered user-friendly due to the fact that they include multiple stages during the test. In addition to the development of performance in end-to-end methods, the studies of compactness in multi-stage methods continue. While converting the output numbers of some algorithms into hash codes in the literature is very costly, some algorithms are very suitable for this task. The DL algorithm is thought to be suitable for this task because it can produce sparse output codes due to its natural nature¹⁴.

DL method creates a representation space using training samples. This is the process of representing a signal in a sparse form. This newly created sparse representation almost represents the original signal¹⁵. Since the general purpose of the DL algorithm is to make the most similar visual words with the given training image, each visual word refers to an image patch. For this reason, high-resolution images are usually divided into patches and used as patches in DL¹⁶. The DL method, which creates different dictionaries for multi-modal objects, can create successful representation for multiple classes. The patch-based representation of the DL method is not suitable for the CBIR task. Also, well-known problems of DL must be overcome¹⁷. Although various solutions have been presented in the literature for dictionary initialization among these problems¹⁸, the search for effective solutions for the problem-specific initial loading continues.

In this study, a hash code generation technique using the representation power of sparse vectors in the dictionary learning method is presented. The proposed method does not require any additional vector down-sampling or hash vector extraction steps. The problem of the initialization of dictionaries, one of the two most important steps of the DL method, and patch-based DL training problems for CBIR is solved in this study. First, a CNN architecture (ResNet50) is trained with original images. Some feature matrices are taken randomly from the middle layer containing the mid-level features of the CNN architecture. Then, these random feature matrices turn to vectors. Dictionaries are created with these vectorized features. The vector in CNN output is used as DL input. Using a complex background dataset containing 26 distinct classes, CBIR experiments are quite challenging for a conventional DL. Because similar image backgrounds for different images make the DL algorithm easily misleading. However, in this study, a feature vectors are used as the input image. Therefore, visual complexity disappears. This is one of the most important contributions of this study. On the other hand, there is no need for additional normalization for dictionary initialization. Another advantage of using features from CNN layers is that the number of iterations can be selected less. This is the second most important contribution. Since each dictionary strongly represents one class, it is assumed that they are similar in sparse codes belonging to the same class. For this reason, it would be very efficient to use these codes as hash codes.

The rest of this study is organized as follows: Section 2 presents the details of the proposed CNN-based DL framework, Section 3 includes dataset details and experimental results. The conclusion is given in Section 4.

2. Proposed Method

Natural images and signals contain a lot of information, depending on the objects or properties they contain. The process of collecting relevant data from this information is quite challenging. On the other hand, collecting all the information is not efficient. The ideal way to process data is to create a sparse representation that can capture most of the signal in a few coefficients. Although many methods that extract the features of the images are used in the literature, the search for a model applied to all images without any modification to be a challenging goal. DL methods are intended to create a discrete state of each image with the help of matrixes called ‘dictionary’. Let there be a matrix called I . Let the I matrix consist of $\{p_i\}_{i=1}^n$ vectors. Here, the parameter p represents the vectors of each image. Therefore, I refers to the dataset. The optimization problem for a DL approach using the K-singular value decomposition (K-SVD) approach is as in Equation 1.

$$(D, \lambda) = \arg \min_{D, \lambda} \|I - D\lambda\|_F^2, \quad \|\lambda_i\|_0 \leq T_0 \quad \forall i \quad (1)$$

where, λ_i represents i th column of the λ , I is image matrix with n pieces of p_i , p_i indicates image vector, D indicates the dictionary matrix, λ represents sparse representations, and T_0 is the sparsity parameter. Here, sub index F represents Frobenius Norm, $\|A\|_F = \sqrt{\sum_{ij} A_{ij}^2}$. Many algorithms are proposed to strengthen the relationship between sparse codes and dictionaries and to solve the optimization problem¹⁹. K-SVD is used in this study. DL approach cannot identify all objects in the image. In addition, it cannot distinguish between foreground objects and background objects. For this reason, training is usually done in patches. In the retrieval process, dominant objects or special situations are investigated in images with a complex background. In some cases, there are small objects in a very dominant and continuous background. In such cases, the searched object in the sparse representation vector cannot be highlighted. An effective solution is presented in this study to overcome this drawback. CNN architectures are used in our framework both to identify objects and to deal with complex backgrounds. CNN is the most effective automatic image analysis method accepted today. It can automatically extract image features. For this process, it uses convolution, pooling, ReLU, dropout, normalization, and fully connected layers. Detailed information about these layers can be found in³. ResNet-50 is used as CNN architecture in this study. Using Equation 2, one CNN output including convolution, pooling, and ReLU can be calculated.

$$f(l_{next}) = pool_{n \times n}(\sigma(w \otimes [D_{in}] + b)) \quad (2)$$

where, l_{next} represents the input, $pool$ indicates the maximum pooling layer, n represents pooling window, σ is ReLU function, w represents convolution kernel, D_{in} represents the input, b is the bias. ResNet-50 architecture is trained using images in all data and labels of these images. A pre-trained ResNet-50 architecture is used in the training process. The vectors obtained at the CNN output are used as the input of the DL structure. In this way, there is no need to train DL with patches. Since there is only one vector for each image, DL encodes the image related vectors directly. There is no complexity for DL as the CNN architecture only sends essential objects to the output vector. DL generates a sparse code (λ) for each image vector. The idea of using these λ as hash code is pretty impressive. Because these discrete codes can represent a class with a specific D . For this reason, it can be said that λ vectors belonging to similar classes are similar. Finally, λ vectors are converted into vectors with fewer parameters by down-sampling. Then binarization is performed.

Another contribution of the proposed method is the initialization of dictionaries. There are several approaches in the literature to assign the first parameters in dictionaries. The initialization process generally contributes positively to DL performance. These values are updated continuously during the DL education process. The purpose of the update process is that the atoms in the dictionary come to the closest representation to the image. λ and dictionary atoms are updated continuously. Another advantage of using CNN is utilized in this study. Since features in the

middle layers of the CNN architecture represent the image, they can be used as dictionary vectors. In this way, the DL training process can be completed by using less number of iterations. Feature matrices are randomly selected from the convolution layers in the middle parts of the ResNet-50 architecture. These feature matrices are converted into vectors and made available to be dictionary vectors. Since these vectors represent the images belonging to the classes in the mid-level, λ learning is more powerful. As this situation indirectly affects the learning of D , it contributes to overall DL performance. The proposed framework is shown in Figure 1.

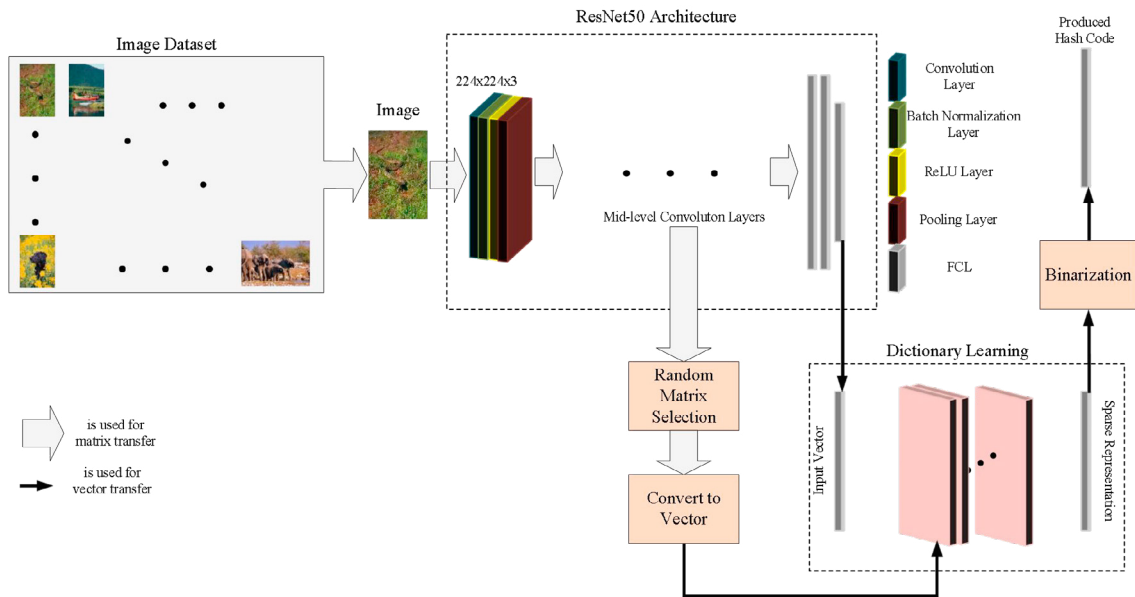


Fig. 1. Proposed Framework.

3. Experimental Setup and Results

3.1. Dataset

The COREL dataset is used to evaluate the performance of the proposed framework²⁰. This dataset contains a total of 10800 images in 80 different categories. All images in the dataset are in RGB color space, the resolutions of these images vary as 120x80 pixels and 80x120 pixels. In this study, not all of the original data is used. 26 classes (classes like dinosaurs, cocktails, planes, sports, human faces, balloons, cars, buses, cats, dogs) are selected from the original dataset. All selected images are used in RGB color space, but their resolution is changed to 224x224 pixels.

3.2. Experimental Results

The proposed method is trained on a computer with Intel Core i7-7700K CPU, 32 GB DDR4 RAM, and NVIDIA GeForce GTX 1080 graphic card. First of all, pre-trained ResNet-50 architecture is used. This ResNet-50 architecture is trained with modified COREL dataset images. Accordingly, an additional FCL layer of 26 neurons is added to the ResNet-50 architecture after the FCL layer of 1000 neurons. During fine-tuning, the mini-batch is selected as size 20. Also, the initial learning rate is used as 0.0005, and the maximum epoch is 15. FCL layer consisting of 1000 neurons is used as DL input. For this reason, one vector with 1000 parameters appears for each image. The combination of all these vectors creates a dataset, and this dataset is the DL input dataset. Convolution layers between 92nd and 157th layers of the ResNet-50 architecture are used to create the dictionaries. Matrices randomly taken from these layers are vectorized within the boundaries of dictionary atoms. These vectors are added

to the related dictionary according to the class information. The length of the λ is the length of the hash code. Changing the length of this vector after calculated by DL degrades performance considerably. Because this is a sparse representation produced by DL. In this study, the hash code length is used as 32 bits.

In order to test the performance of the proposed framework, the lambda vectors in the DL output are used in their original form. In this state, comparisons of each vector to each other are made in the Euclidean space. Accordingly, intra-class similarity rates and inter-class differences are measured. The results produced by the original DL architecture are used to compare the performance of the two proposed frameworks. The results obtained from the experiments performed are shown in Table 1. It should be noted that the results in Table 1 are not results obtained with hash codes, they are obtained with original λ s. The parameter expressed by P@N represents the mean average precision (mAP) value for N vector retrieval.

Table 1. The retrieval performance of the proposed framework according to Euclidean distance

Method	P@5	P@10	P@15	P@20	P@25	P@30	P@35	P@40	P@45	P@50
DL	68,05	64,98	58,44	54,37	53,82	51,99	48,56	46,59	42,57	42,25
Proposed Framework	91,96	85,50	81,17	77,67	74,75	72,04	69,50	67,04	64,49	62,14

As can be seen in Table 1, as the number of images to be retrieved increases, the mean average precision value decreases. In other words, this means that as you move away from the classroom center, the difference with other classes decreases (it should not be forgotten that there are approximately 100-150 images in each class). Another issue that should not be ignored in the experiments performed with conventional DL is the tolerance range of the performance. The average performance deviation in 5-fold experiments is in the range of 7-9%. As a second step, the codes in the DL output are converted into binary. In the process of converting to binary, a very simple way is followed. The mean value of each vector is calculated and this mean value is the threshold for that vector. In this way, all vectors are converted to binary and a hash table is created. Experiments performed with hash codes in hamming space are shown in Table 2.

Table 2. The retrieval performance of the proposed framework according to hamming distance

Method	P@5	P@10	P@15	P@20	P@25	P@30	P@35	P@40	P@45	P@50
DL	60,64	51,60	50,64	47,12	44,08	43,85	43,61	42,68	40,31	38,86
Proposed Framework	86,62	80,61	76,74	73,64	71,09	68,52	66,09	63,71	61,08	58,61

When Table 1 and Table 2 are compared, it is observed that the hamming performance is slightly lower. It is possible to compensate for this performance degradation by using more efficient binarization algorithms. This study aims to show the effective use of the DL algorithm for retrieval. For this reason, binarization is not considered. On the other hand, although there is a slight decrease in mAP performance, there is faster processing in the hamming space in the retrieval time part. Table 3 shows the retrieval time comparison.

Table 3. Average retrieval time of proposed framework according

Method	P@25
DL	26,7 ms
Proposed Framework	25,2 ms

4. Conclusion

This study provides an efficient CBIR framework for datasets containing a small number of tagged data. It solves the training problem with a small number of labeled data using the dictionary learning approach. Another reason for choosing the DL approach is the thought that lambdas can be vectors with similar numerical ratios for similar images. So you can convert lambdas to hash codes without much effort. However, the problems of DL methods prevented obtaining the desired high performance. Thanks to the CNN supported DL architecture proposed in this

study, it is provided to add DL to retrieval tasks quite simply and effectively. In order to prove the effectiveness of the proposed framework, comparisons are made in Euclidean space first. The contribution made according to the mAP results obtained in the Euclidean space is very promising. On the other hand, a simple binarization approach is used to compare hash code results. In future studies, as a continuation of this study, more effective binarization and hash tables will be studied, and faster retrieval will be studied.

Acknowledgment

This research is funded by the Scientific and Technological Research Council of Turkey (TÜBİTAK) under grant number 120E018.

References

1. Wang J, Zhang T, Song J, Sebe N, Shen HT. A Survey on Learning to Hash. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 2018;40(4):769-90.
2. Jiang Q-Y, Cui X, Li W-J. Deep Discrete Supervised Hashing. *IEEE Transactions on Image Processing*. 2018;27(12):5996-6009.
3. Öztürk Ş. Stacked auto-encoder based tagging with deep features for content-based medical image retrieval. *Expert Systems with Applications*. 2020;161.
4. Öztürk Ş. Siamese Network kullanarak İki Aşamalı Sıralı Kayıplara dayalı Otomatik Hash Kodu Üretimi. *European Journal of Science and Technology*. 2020.
5. Arun KS, Govindan VK, Kumar SDM. Enhanced bag of visual words representations for content based image retrieval: a comparative study. *Artificial Intelligence Review*. 2019;53(3):1615-53.
6. Öztürk Ş, Akdemir B. Application of Feature Extraction and Classification Methods for Histopathological Image using GLCM, LBP, LBGLCM, GLRLM and SFTA. *Procedia Computer Science*. 2018;132:40-6.
7. Bai X, Yang M, Huang T, Dou Z, Yu R, Xu Y. Deep-Person: Learning discriminative deep features for person Re-Identification. *Pattern Recognition*. 2020;98.
8. Chaudhuri U, Banerjee B, Bhattacharya A. Siamese graph convolutional network for content based remote sensing image retrieval. *Computer Vision and Image Understanding*. 2019;184:22-30.
9. Husain SS, Bober M. REMAP: Multi-Layer Entropy-Guided Pooling of Dense CNN Features for Image Retrieval. *IEEE Transactions on Image Processing*. 2019;28(10):5201-13.
10. Roy S, Sangineto E, Demir B, Sebe N. Metric-Learning-Based Deep Hashing Network for Content-Based Retrieval of Remote Sensing Images. *IEEE Geoscience and Remote Sensing Letters*. 2020:1-.
11. Liu H, Wang R, Shan S, Chen X. Deep Supervised Hashing for Fast Image Retrieval. *International Journal of Computer Vision*. 2019;127(9):1217-34.
12. Sezavar A, Farsi H, Mohamadzadeh S. Content-based image retrieval by combining convolutional neural networks and sparse representation. *Multimedia Tools and Applications*. 2019;78(15):20895-912.
13. Yue-Hei Ng J, Yang F, Davis LSJae-p. Exploiting Local Features from Deep Networks for Image Retrieval 2015 April 01, 2015:[arXiv:1504.05133 p.]. Available from: <https://ui.adsabs.harvard.edu/abs/2015arXiv150405133Y>.
14. Xu X-S. Dictionary Learning Based Hashing for Cross-Modal Retrieval. *Proceedings of the 2016 ACM on Multimedia Conference - MM '16* 2016. p. 177-81.
15. Deng C, Tang X, Yan J, Liu W, Gao X. Discriminative Dictionary Learning With Common Label Alignment for Cross-Modal Retrieval. *IEEE Transactions on Multimedia*. 2016;18(2):208-18.
16. Öztürk Ş, Akdemir B. Phase classification of mitotic events using selective dictionary learning for stem cell populations. *Computers & Electrical Engineering*. 2018;67:25-37.
17. Jung A, Eldar YC, Görtz NJae-p. Performance Limits of Dictionary Learning for Sparse Coding 2014 February 01, 2014:[arXiv:1402.4078 p.]. Available from: <https://ui.adsabs.harvard.edu/abs/2014arXiv1402.4078J>.
18. Rusu C, Dumitrescu B. An initialization strategy for the dictionary learning problem. *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* 2014. p. 6731-5.
19. Liu J, Liu W, Li Q, Ma S, Chen G. Evaluation of K-SVD with different embedded sparse representation algorithms. *2016 12th International Conference on Natural Computation, Fuzzy Systems and Knowledge Discovery (ICNC-FSKD)* 2016. p. 426-32.
20. Dacheng T, Xiaou T, Xuelong L, Xindong W. Asymmetric bagging and random subspace for support vector machines-based relevance feedback in image retrieval. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 2006;28(7):1088-99.